Linux Kernel Programming

Linux Kernel programming

Agenda

- SPI Driver
- PCI Driver

SPI Driver – Introduction

- The "Serial Peripheral Interface" (SPI) is a synchronous four wire serial link used to connect microcontrollers to sensors, memory, and peripherals introduced by Motorola. It's a simple "de facto" standard, not complicated enough to acquire a standardization body. SPI uses a master/slave configuration.

- The three signal wires hold a clock (SCK, often on the order of 10 MHz), and parallel data lines with "Master Out, Slave In" (MOSI) or "Master In, Slave Out" (MISO) signals. (Other names are also used.) There are four clocking modes through which data is exchanged; mode-0 and mode-3 are most commonly used. Each clock cycle shifts data out and data in; the clock doesn't cycle except when there is a data bit to shift. Not all data bits are used though; not every protocol uses those full duplex capabilities.

- SPI masters use a fourth "chip select" line to activate a given SPI slave device, so those three signal wires may be connected to several chips in parallel. All SPI slaves support chipselects; they are usually active low signals, labeled nCSx for slave 'x' (e.g. nCS0). Some devices have other signals, often including an interrupt to the master.

SPI Driver – Introduction

- Unlike serial busses like USB or SMBus, even low level protocols for SPI slave functions are usually not interoperable between vendors (except for commodities like SPI memory chips).
  - SPI may be used for request/response style device protocols, as with touchscreen sensors and memory chips.
  - It may also be used to stream data in either direction (half duplex), or both of them at the same time (full duplex).
  - Some devices may use eight bit words. Others may use different word lengths, such as streams of 12-bit or 20-bit digital samples.
  - Words are usually sent with their most significant bit (MSB) first, but sometimes the least significant bit (LSB) goes first instead.
  - Sometimes SPI is used to daisy-chain devices, like shift registers.

- In the same way, SPI slaves will only rarely support any kind of automatic discovery/enumeration protocol. The tree of slave devices accessible from a given SPI master will normally be set up manually, with configuration tables.

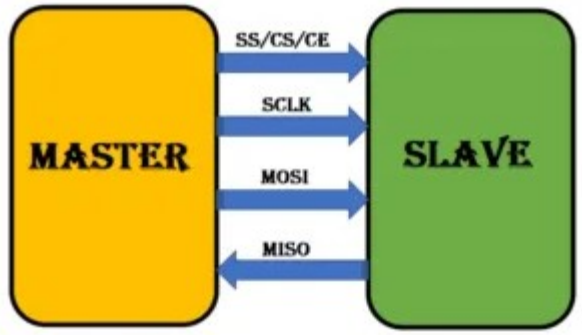
SPI Driver – Introduction

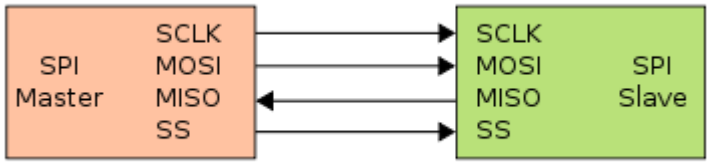
- SPI is only one of the names used by such four-wire protocols, and most controllers have no problem handling "MicroWire" (think of it as half-duplex SPI, for request/response protocols), SSP ("Synchronous Serial Protocol"), PSP ("Programmable Serial Protocol"), and other related protocols.

- Some chips eliminate a signal line by combining MOSI and MISO, and limiting themselves to half-duplex at the hardware level. In fact some SPI chips have this signal mode as a strapping option. These can be accessed using the same programming interface as SPI, but of course they won't handle full duplex transfers. You may find such chips described as using "three wire" signaling: SCK, data, nCSx. (That data line is sometimes called MOMI or SISO). Microcontrollers often support both master and slave sides of the SPI protocol.

SPI Driver – Introduction

- SPI needs 4 wires at least. But some of the recent day devices support 3 pin mode and 4pin modes also.
  - MOSI – Master Out Slave In
  - MISO – Master In Slave Out
  - SCLK – Serial Clock
  - SS/CS/CE – Salve Select/Chip Select/Chip Enable



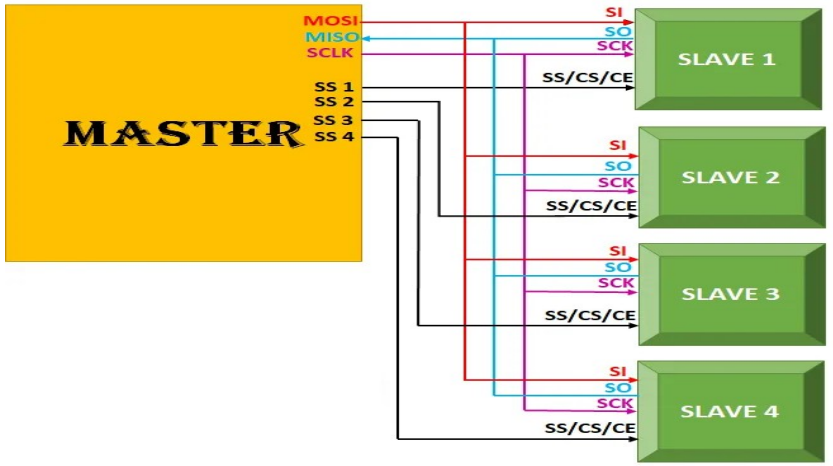


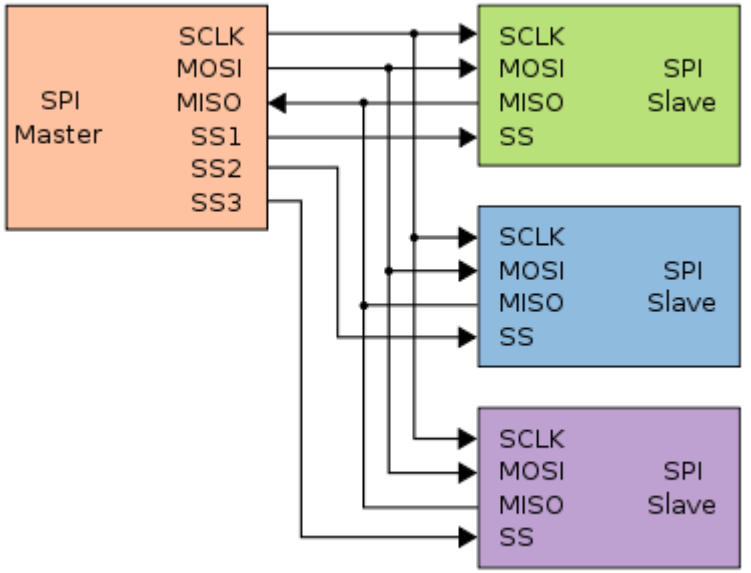
SPI Driver – Introduction

- SPI protocol is a Full-duplex as this protocol has separate pins for incoming data and outgoing data. So, we can transmit and receive the data at the same time.

- on a slave-only device, MOSI may be labeled as SDI (Serial Data In) and MISO may be labeled as SDO (Serial Data Out).

- When you increase the number of slaves, then the Slave Select pin also increases.





SPI Driver – Introduction

- In above picture the master device has to use the 7 pins for 4 slave devices. This is one of the disadvantages. In I2C, the number of pins won't increase even if you increase the slave devices. To avoid this scenario, they have introduced daisy chain logic.

- The communication (data transmission between the master and the slave) is synchronized to the clock generated by the master. So, as it is synchronized using the clock, it doesn't have to wait for a response before doing other operations. SPI devices support much higher clock frequencies compared to I2C interfaces.

- Linux developers using SPI are probably writing device drivers for embedded systems boards. SPI is used to control external chips, and it is also a protocol supported by every MMC or SD memory card. (The older "DataFlash" cards, predating MMC cards but using the same connectors and card shape, support only SPI.) Some PC hardware uses SPI flash for BIOS code.

- SPI slave chips range from digital/analog converters used for analog sensors and codecs, to memory, to peripherals like USB controllers or Ethernet adapters; and more.

SPI Driver – Introduction

- Most systems using SPI will integrate a few devices on a mainboard. Some provide SPI links on expansion connectors; in cases where no dedicated SPI controller exists, GPIO pins can be used to create a low speed "bitbanging" adapter. Very few systems will "hotplug" an SPI controller; the reasons to use SPI focus on low cost and simple operation, and if dynamic reconfiguration is important, USB will often be a more appropriate low-pincount peripheral bus.

- Many microcontrollers that can run Linux integrate one or more I/O interfaces with SPI modes. Given SPI support, they could use MMC or SD cards without needing a special purpose MMC/SD/SDIO controller.

SPI Driver – Introduction

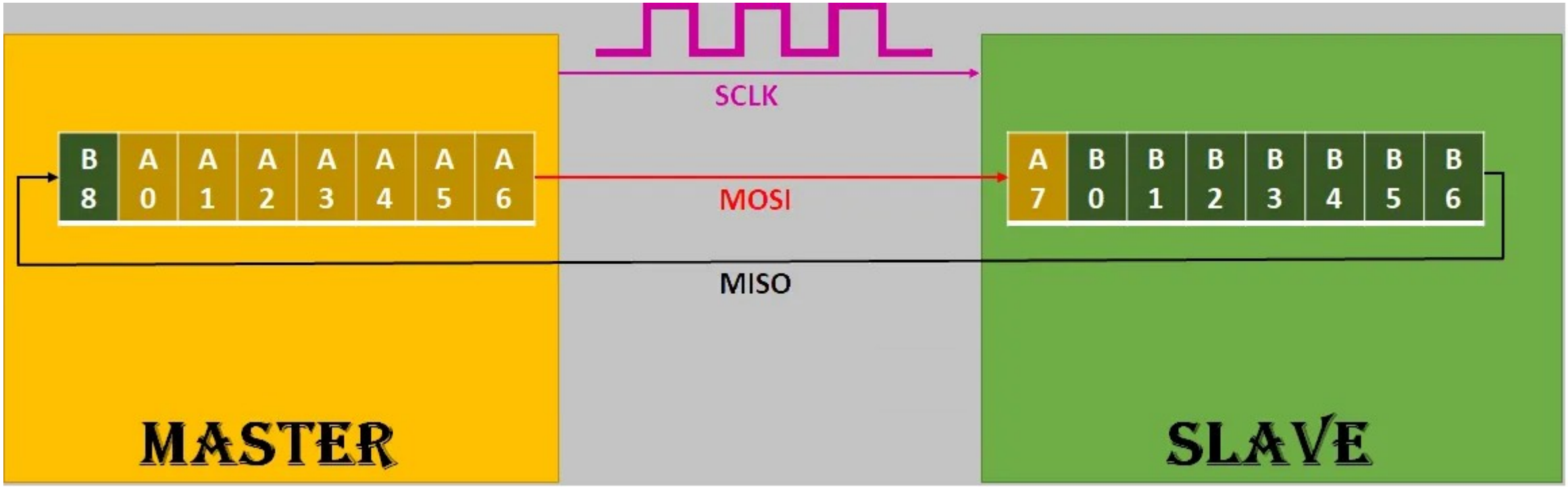
- To begin communication, the bus master configures the clock, using a frequency supported by the slave device, typically up to a few MHz. Once the clock has been configured, then the master makes the Slave select (SS/CS) pin LOW that initiates the transfer. When multiple slaves are used, the master has to make the respective slave's Slave select (SS/CS) pin LOW.

- During each SPI clock cycle, full-duplex data transmission occurs. The master sends a bit on the MOSI pin and the slave reads it, while the slave sends a bit on the MISO pin and the master reads it. This sequence is maintained even when only one-directional data transfer is intended.

- Transmissions normally involve two shift registers of eight bits (mostly but size varies based on the slaves), one in the master and one in the slave. Data is usually shifted out with the most significant bit (MSB) first. On the clock edge, both master and slave shift out a bit and output it on the transmission line to the counterpart. On the next clock edge, at each receiver, the bit is sampled from the transmission line and set as a new least-significant bit of the shift register. After the register bits have been shifted out and in, the master and slave have exchanged register values. If more data needs to be exchanged, the shift registers are reloaded and the process repeats. Transmission may continue for any number of clock cycles.

SPI Driver – Introduction

- When the master completes the data transfer, then the master stops toggling the clock signal and it makes the SS/CS pin HIGH.



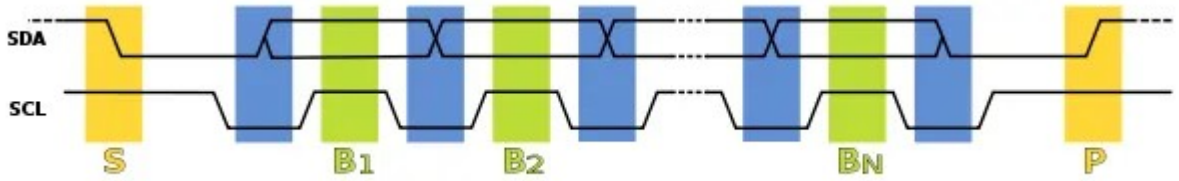
SPI Driver – Introduction

- **SPI Modes** - The SPI interface defines no protocol for data exchange, limiting overhead and allowing for high-speed data streaming. We saw how the data is being transferred to the slave from the master. As we have an understanding of data communication, let's go a little bit deeper. In addition to setting the clock frequency, the master must also configure the clock polarity and phase with respect to the data. Clock polarity (CPOL) and clock phase (CPHA) are the two parameters that are used to form four unique modes to provide flexibility in communication between master and slave.

CPOL	CPHA	MODE
0	0	MODE 0
0	1	MODE 1
1	0	MODE 2
1	1	MODE 3

SPI Driver – Introduction

- Why Different modes.



- In the above image, the yellow color is START and STOP condition. Blue colors are where the data is being changed when SCL is Low. And the Green colors are where the data is read by the slave or master when SCL is High. So, these data changing and sampling place is fixed in the I2C. But in SPI, it is not fixed. It is flexible and the CPOL and CPHA are used to tell the master and slave that when to sample the data. I guess you must be clear now.

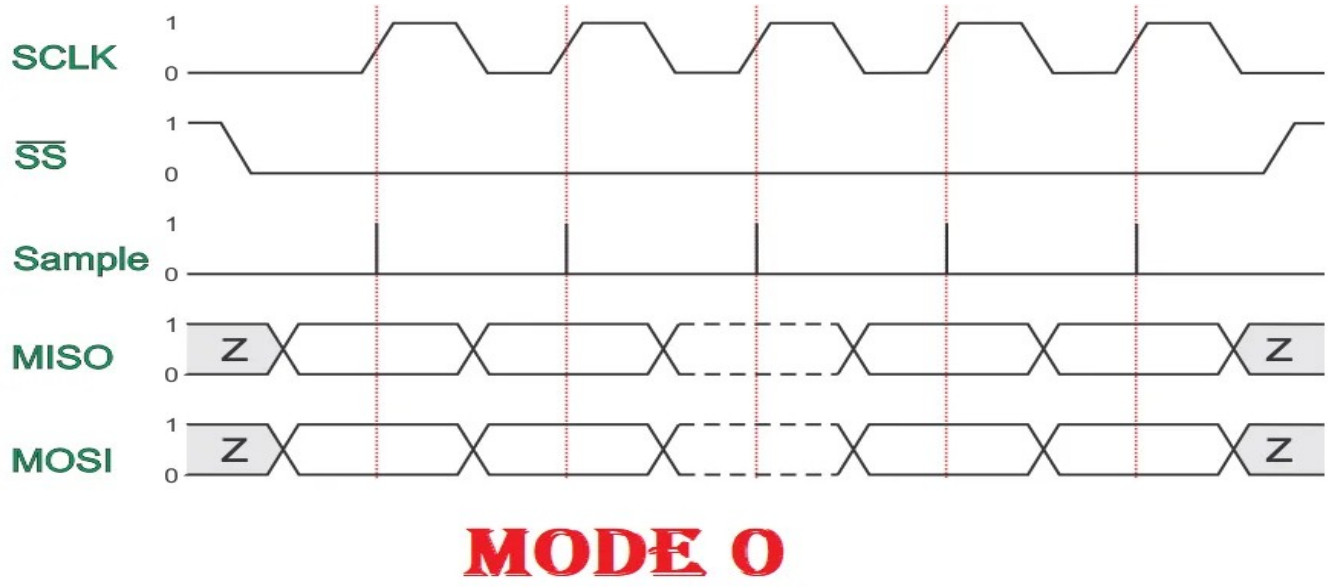
- The clock polarity (CPOL) determines the initial state of the Clock pin (SCLK). If CPOL is 0 means then the SCLK line's initial state is Low, and if CPOL is 1 means the SCLK line's initial state is High.

- The clock phase (CPHA) determines the edge. If CPHA is 0 then it means the leading edge (1st edge), and if CPHA is 1, then it means the trailing edge (2nd edge). These edges can be high to low or low to high.

SPI Driver – Introduction

Mode 0

- In this mode, CPOL is 0 and CPHA is also 0. So, the SCLK starts as Low and the data will be sampled at the first edge.

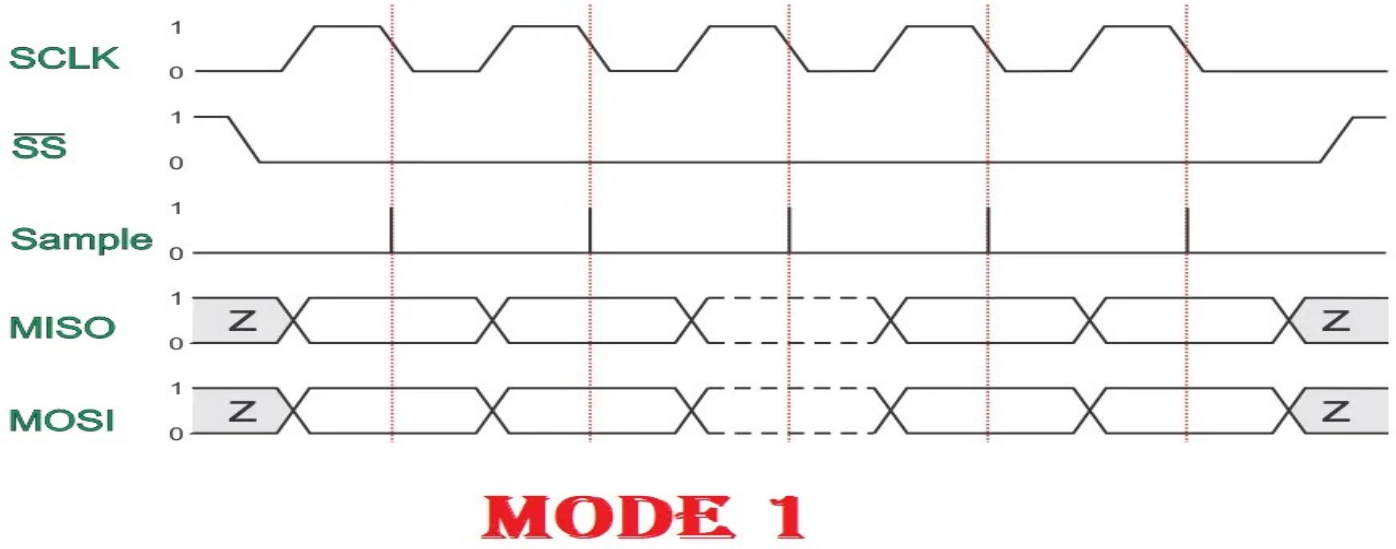


- The data must be available before the first clock signal rising. The clock idle state is LOW. The data on MISO and MOSI lines must be stable while the clock is high and can be changed when the clock is low. The data is captured on the clock's low-to-high transition and can be propagated on high-to-low clock transition.

SPI Driver – Introduction

Mode 1

- In this mode, CPOL is 0 and CPHA is also 1. So, the SCLK starts as Low and the data will be sampled at the second edge. Check the below image.

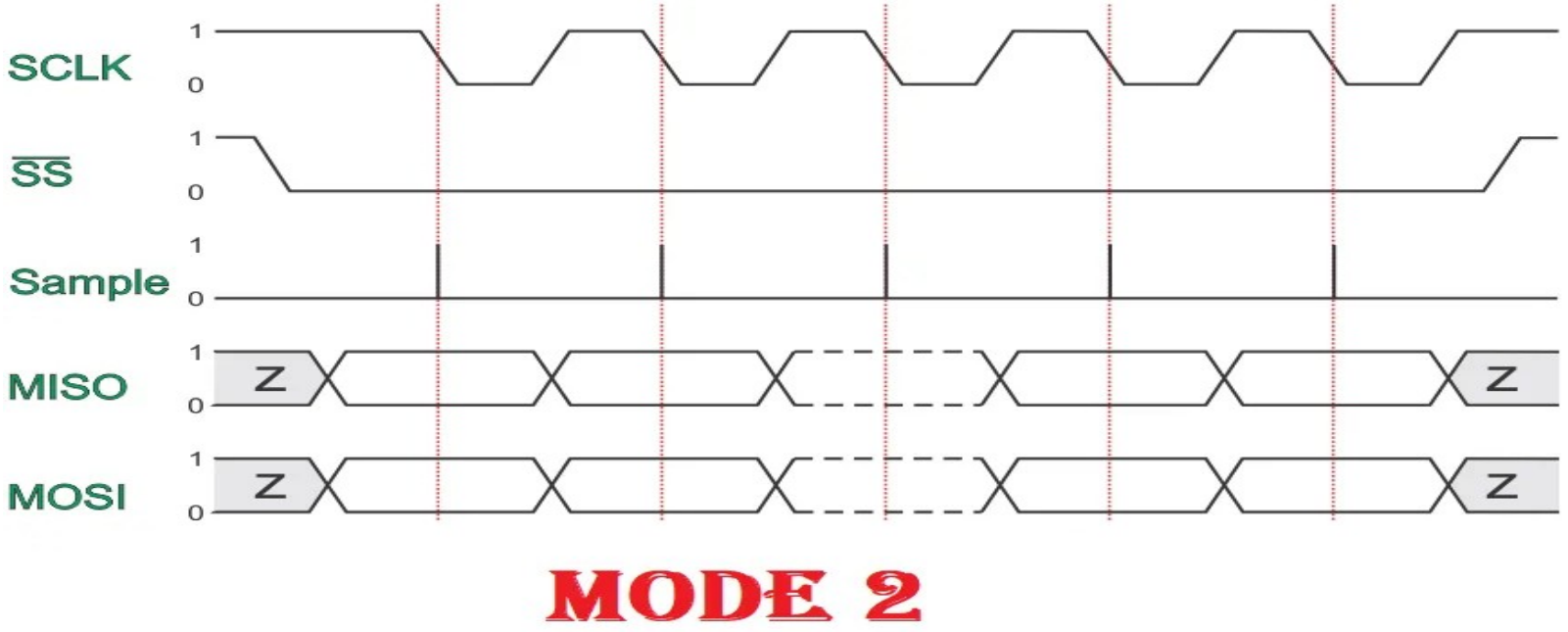


- The first clock signal rising can be used to prepare the data. The clock idle state is LOW. The data on MISO and MOSI lines must be stable while the clock is low and can be changed when the clock is high. The data is captured on the clock's high-to-low transition and propagated on low-to-high clock transition.

SPI Driver – Introduction

Mode 2

- In this mode, CPOL is 1 and CPHA is also 0. So, the SCLK starts as High and the data will be sampled at the first edge. Check the below image to understand that.

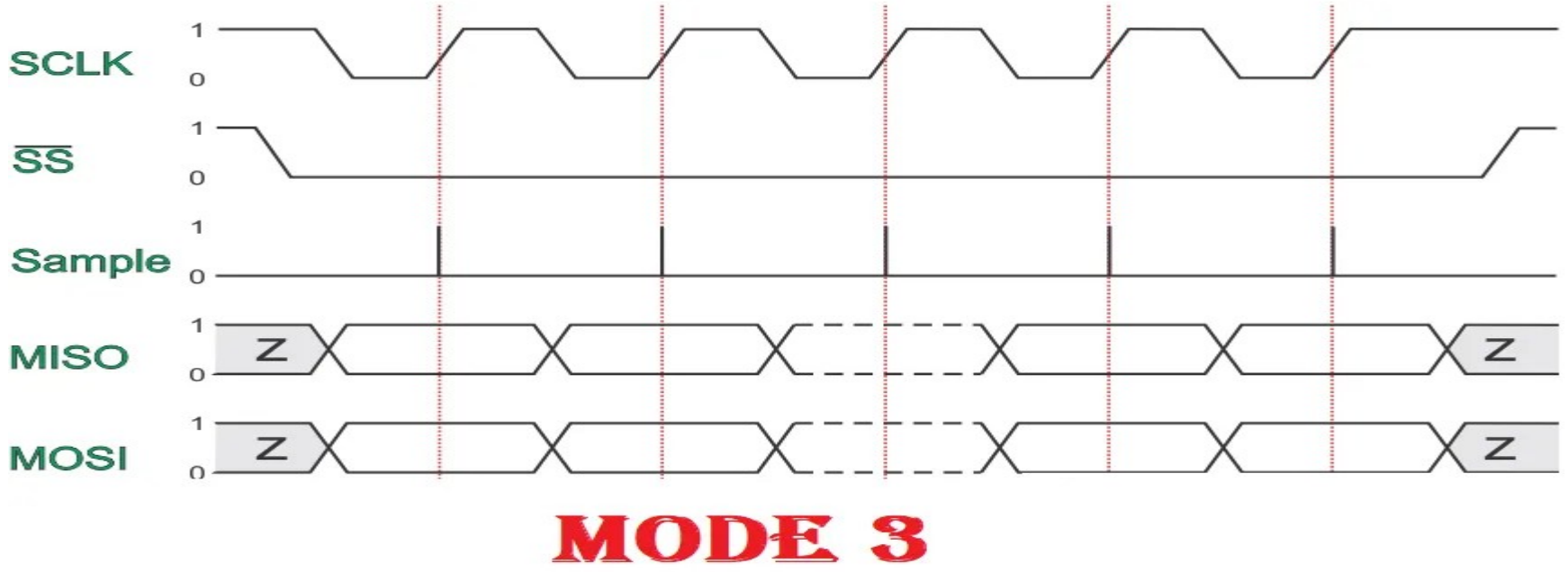


- The data must be available before the first clock signal falling. The clock idle state is HIGH. The data on MISO and MOSI lines must be stable while the clock is low and can be changed when the clock is high. The data is captured on the clock's high-to-low transition and propagated on low-to-high clock transition.

SPI Driver – Introduction

Mode 3

In this mode, CPOL is 1 and CPHA is also 1. So, the SCLK starts as High and the data will be sampled at the second edge. Check the below image to understand that.

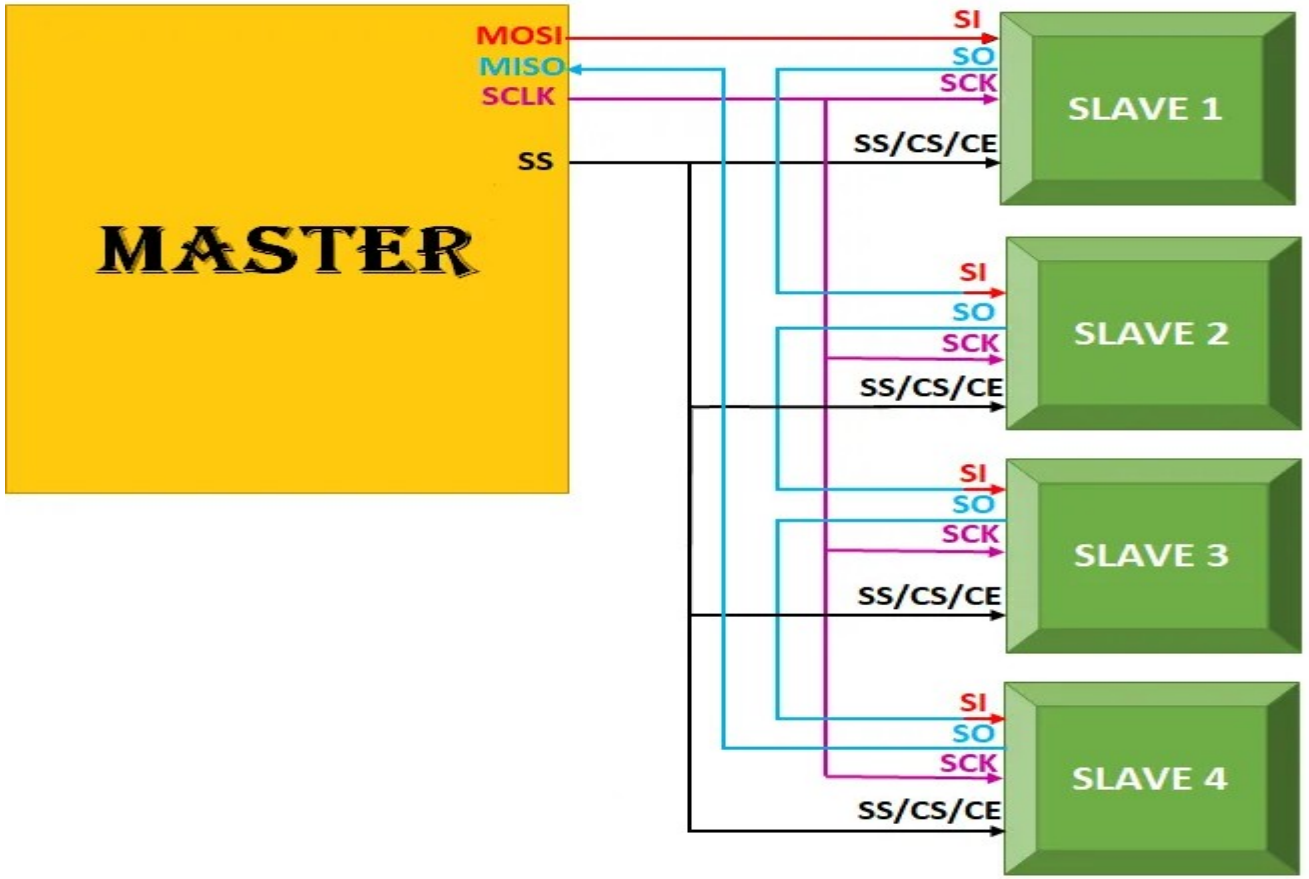


The first clock signal falling can be used to prepare the data. The clock idle state is HIGH. The data on MISO and MOSI lines must be stable while the clock is high and can be changed when the clock is low. The data is captured on the clock's low-to-high transition and propagated on high-to-low clock transition.

SPI Driver – Introduction

Daisy Chain

In the Daisy chain method, we can use a single SS/CS line for all the slaves.



SPI Driver – Introduction

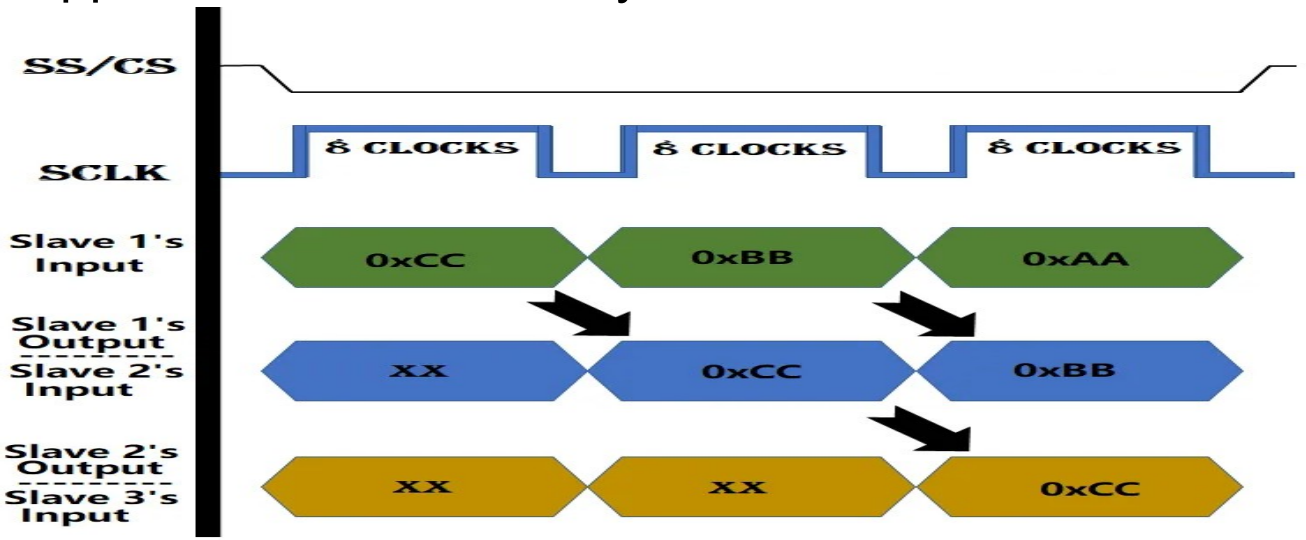
- We made two modifications to the normal SPI connections.
- Removed the other SS/CS pins and connected all the slave's SS/CS pin to the one SS/CS pin of the master.
- All the slave's Output (SO) is connected to the next slave's input (SI) and the last slave's output is connected to the master's MISO.

SPI Driver – Introduction

Daisy Chain working

- Slave 1's input (SI) is directly connected to the master's MOSI. When the master transmits the data, it will be stored in slave 1's internal shift register for the first 8 clock cycles. Then in the next 8 clock cycles, slave 1 will be sent out the data from its internal shift register through its output (SO). As Slave 1's output (SO) line is connected to Slave 2's input (SI), Slave 2 will receive the data in the next 8 clock cycles. And this process continues as long as the master generating the clocks and SS/CS line is Low.

- Assume that the master wants to send 0xAA to slave 1, 0xBB to slave 2, and 0xCC to slave 3. The below picture explains what happens in each 8 clock cycle.



SPI Driver – Introduction

- In this method, as data is propagated from one slave to the next, the number of clock cycles required to transmit data is proportional to the slave position in the daisy chain. In the above example, if the master wants to send the 8-bits of data to Slave 3, then it takes 24 clock cycles. If you have used the normal SPI mode instead of the daisy chain, then the master would just take 8 clock cycles to send 8-bit data instead of the 24 clock cycles.

- Daisy-chain mode is not necessarily supported by all SPI devices.

SPI Driver – Introduction

- Advantages of SPI
  - It's way faster than asynchronous serial communication and I2C (almost twice as fast).
  - No start and stop bits, so the data can be streamed continuously without interruption.
  - No slave addressing mechanism like I2C.
  - It has separate MISO and MOSI lines, so data can be sent and received at the same time (Full duplex).
  - Not Limited to 8-bit data.
  - The received hardware (slave) can be a simple shift register.
  - It supports multiple slave devices.

SPI Driver – Introduction

- Disadvantages of SPI
  - It requires more lines (wires) than other communication methods. (UART and I2C only require 2 lines).
  - There is no acknowledgment to inform that data has been successfully transferred like I2C and No error detection protocol is defined.
  - The communications must be well-defined in advance (you can't send random amounts of data whenever you want).
  - The master must control all communications (slaves can't talk directly to each other).
  - It usually requires separate CS lines to each peripheral, which can be problematic if numerous slaves are needed.
  - SPI only supports a single master.
  - Can be used only from short distances.

SPI Driver – Introduction

- Applications
  - SPI is used to talk to a variety of peripherals, such as,
  - Sensors: temperature, pressure, ADC, touchscreens, video game controllers
  - Control devices: audio codecs, digital potentiometers, DAC
  - Camera lenses: Canon EF lens mount
  - Communications: Ethernet, USB, USART, CAN, IEEE 802.15.4, IEEE 802.11, handheld video games
  - Memory: flash and EEPROM
  - Real-time clocks
  - LCD
  - Any MMC or SD card

SPI Driver – Introduction

I2C	SPI
I2C has been developed in 1982 by Philips Semiconductors (NXP).	SPI is developed by Motorola in the mid-1980s.
It supports multi masters.	It supports only one master.
It needs a maximum of 2 wires.	It needs a minimum of 4 wires and the maximum is based on the number of slaves.
It is half-duplex communication as it has only one wire for Tx and Rx.	It is full-duplex communication as it has separate wires for Rx and Tx.
It is slower than the SPI.	It is faster than the I2C.

SPI Driver – Introduction

I2C	SPI
I2C has an acknowledgment mechanism.	This doesn't have an acknowledgment mechanism.
In this protocol, the slave has to have a unique address.	In this protocol, the slave doesn't need an address. Master will use the SS/CS wire to select the slaves.
Due to Start, Stop, Address bits, it has overhead.	But it will directly start the data transfer. Start and Stop are not required. So, no overhead here.
I2C is better for long-distance.	SPI is better for a short distance.

SPI Driver – Implementation

- To enable SPI 1, please add the below line in the /boot/config.txt

  dtoverlay=spi1-1cs,cs0_spidev=disabled

- The above line enables the SPI 1 and it disables the SPIDev application. So that, we can use the SPI 1 bus. You can refer to this if you have any doubt.

SPI Driver – Implementation

- The SPI device driver in Linux is mainly managed by the SPI subsystem, and it is divided into 3 sections.
  - SPI Core
  - SPI Controller Driver
  - SPI Protocol Driver

SPI Driver – Implementation

SPI Core

- The SPI core provides APIs for the definition of core data structures, registration, cancellation management of SPI controller drivers and device drivers. It is a hardware platform-independent layer, that shields the differences of the physical bus controller downwards and defines a unified access strategy and interface. It provides a unified interface upwards. So that, the SPI device driver can send and receive data through the SPI bus controller. In Linux kernel, the SPI core code is located in kernel/drivers/spi/spi.c.

SPI Driver – Implementation

SPI Controller Driver

- The SPI Controller driver is the platform-specific driver. So, each SoC manufactures has to write this driver for their platform or MCU. controllers These SPI controller drivers may be built into System-On-Chip processors, and often support both Master and Slave roles. These drivers touch hardware registers and may use DMA or they can be GPIO bit bangers, needing just GPIO pins. Its responsibility is to implement a corresponding read and write a method for each SPI bus in the system. Physically, each SPI controller can connect several SPI slave devices. When the system is turned on, the SPI controller driver is loaded first. A controller driver is used to support the reading and writing of a specific SPI bus. So, this SPI controller driver is similar to the adapter/bus driver in the I2C.

- You can see the all SPI controller driver in kernel/drivers/spi/.

SPI Driver – Implementation

- Controller drivers are used to abstract anddrive transactions on an SPI master.
- The host SPI peripheral registers are accessed by callbacks provided to the SPI core driver. (drivers/spi/spi.c)
- struct spi_controller

- Allocate a controller
  - **spi_alloc_master()**
- Set controller fields and methods
  - mode_bits - flags e.g. **SPI_CPOL, SPI_CPHA, SPI_NO_CS, SPI_CS_HIGH, SPI_RX_QUAD, SPI_LOOP**
  - **.setup()** - configure SPI parameters
  - **.cleanup()** - prepare for driver removal
  - **.transfer_one_message()/.transfer_one()** - dispatch one msg/transfer (mutually exclusive)
- Register a controller
  - **spi_register_master()**

SPI Driver – Implementation

Controller Devicetree Binding

Example:

```
spi1: spi@481a0000 {
        compatible = "ti,omap4-mcspi";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x481a0000 0x400>;
        interrupts = <125>;
        ti,spi-num-cs = <2>;
        ti,hwmods = "spi1";
        dmas = <&edma 42 0
        &edma 43 0
        &edma 44 0
        &edma 45 0>;
        dma-names = "tx0", "rx0", "tx1", "rx1";
        status = "disabled";
};
```

SPI Driver – Implementation

SPI Protocol Driver

- Each SPI bus controller may connect to multiple slave devices. This driver is used to communicate to specific devices through the SPI bus. This is similar to the I2C client driver.

Slave Node Devicetree Binding

SPI slave nodes must be children of the SPI controller node.

In master mode, one or more slave nodes (up to the number of chip selects) can be present.

Required properties are:

- compatible - Name of SPI device following generic names recommended practice.
- reg - Chip select address of device.
- spi-max-frequency - Maximum SPI clocking speed of device in Hz.

SPI Driver – Implementation

Slave Node Devicetree Binding

All slave nodes can contain the following optional properties:

- spi-cpol - Empty property indicating device requires inverse clock polarity (CPOL) mode.
- spi-cpha - Empty property indicating device requires shifted clock phase (CPHA) mode.
- spi-cs-high - Empty property indicating device requires chip select active high.
- spi-3wire - Empty property indicating device requires 3-wire mode.
- spi-lsb-first - Empty property indicating device requires LSB first mode.
- spi-tx-bus-width - The bus width that is used for MOSI. Defaults to 1 if not present.
- spi-rx-bus-width - The bus width that is used for MISO. Defaults to 1 if not present.
- spi-rx-delay-us - Microsecond delay after a read transfer.
- spi-tx-delay-us - Microsecond delay after a write transfer

SPI Driver – Implementation

Slave Node Devicetree Binding

Example:

```
&spi1 {
        #address-cells = <1>;
        #size-cells = <0>;
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&spi1_pins>;
        myspi@0 {
                compatible = "mycompany,myspi";
                spi-max-frequency = <2000000>;
                spi-cpha;
                ...
                reg = <0>;
        };
...
};
```

SPI Driver – Implementation

Platform Registration

```
struct spi_board_info {
        char modalias;
        const void * platform_data;
        const struct property_entry * properties;
        void * controller_data;
        int irq;
        u32 max_speed_hz;
        u16 bus_num;
        u16 chip_select;
        u16 mode;
};
```

SPI Driver – Implementation

Platform Registration

Example:

```
static struct spi_board_info myspi_board_info[] = {
 {
        .modalias = "myspi",
        .platform_data = &myspi_info,
        .irq = MYIRQ,
         .max_speed_hz = 2000000,
        .chip_select = 2,
        …..
        },
};
```

SPI Driver – Implementation

SPI Driver – Implementation

- Steps that involve writing the SPI protocol device driver are given below.
  - Get the SPI Controller driver
  - Add the slave device to the SPI Controller
  - Configure the SPI
  - Now the driver is ready. So you can transfer the data between master and slave.
  - Once you are done, then remove the device.

SPI Driver – Implementation

Get the SPI Controller driver

- First, you need to get the controller driver. To do that, you can use the below API. Before using that API, you need to know which SPI bus you are using.

  struct spi_controller * spi_busnum_to_master(u16 bus_num)

  where,

  bus_num – the master's bus number

  Return

- It returns a pointer to the relevant spi_controller (which the caller must release), or NULL if there is no such master registered.

SPI Driver – Implementation

```
struct  spi_master *master;
master = spi_busnum_to_master( 1 );
```

SPI Driver – Implementation

Add the slave device to the SPI Controller

- In this step, you need to create a spi_board_info structure that has the slave device's details.

```
struct spi_board_info {
  char modalias[SPI_NAME_SIZE];
  const void      *platform_data;
  const struct property_entry *properties;
  void *controller_data;
  int irq;
  u32 max_speed_hz;
  u16 bus_num;
  u16 chip_select;
  u32 mode;
};
```

SPI Driver – Implementation

where,

modalias – This string is used to identifies the driver.

platform_data – the particular data stored there is driver-specific.

properties – Additional device properties for the device.

controller_data – some controllers need hints about hardware setup, e.g. for DMA.

irq – Interrupt number

max_speed_hz – Maximum speed that slave supports. Refer to the slave's datasheet.

bus_num– SPI bus number that slave is going to connect.

chip_select – Chip select that you want to use.

mode – SPI mode. Refer to the slave's datasheet.

SPI Driver – Implementation

- Once you have filled this structure, then you can use the below API to add the slave device to the SPI controller driver.

struct spi_device * spi_new_device(struct spi_controller *ctlr, struct spi_board_info *chip)

where,

ctlr – Controller to which device is connected. In our case, it has been returned by spi_busnum_to_master().

chip – SPI slave device structure.

Return

- It returns the new device, or NULL.

- This spi_new_device() is enough as this function handles the spi_alloc_device() and spi_add_device() inside.

SPI Driver – Implementation

```
struct spi_board_info test_spi_device_info =
{
  .modalias     = “test-spi-ssd1306-driver",
  .max_speed_hz = 4000000,              // speed your device (slave) can handle
  .bus_num      = SPI_BUS_NUM,          // SPI 1
  .chip_select  = 0,                    // Use 0 Chip select (GPIO 18)
  .mode         = SPI_MODE_0            // SPI mode 0
};
static struct spi_device *test_spi_device;
test_spi_device = spi_new_device( master, &test_spi_device_info );
if( test_spi_device == NULL )
{
  pr_err("FAILED to create slave.\n");
  return -ENODEV;
}
```

SPI Driver – Implementation

Configure the SPI

- Till now, we have added the slave device to the controller driver. If we change any mode or clock frequency, then we have to call this below API to take effect on the bus.

int spi_setup(struct spi_device *spi)

where,

spi – the device whose settings are being modified.

Return

- It returns zero on success, else a negative error code.

SPI Driver – Implementation

Message Transfer

- We have done with our initialization. Now we can transfer the data using the below APIs.

- This function will be blocked until the master completes the transfer.

int spi_sync_transfer(struct spi_device *spi, struct spi_transfer *xfers, unsigned int num_xfers)

where,

spi – device with which data will be exchanged.

xfers – An array of spi_transfers.

num_xfers – Number of items in the xfer array.

Return

It returns zero on success, else a negative error code.

SPI Driver – Implementation

- This API is used to transfer the data asynchronously. This call may be used in_irq and other contexts which can't sleep,

  int spi_async(struct spi_device *spi, struct spi_message *message)

  where,

  spi – device with which data will be exchanged.

  message – describes the data transfers, including completion callback.

  Return

  It returns zero on success, else a negative error code.

SPI Driver – Implementation

- This API is used to write the data and followed by a read. This is synchronous.

int spi_write_then_read(struct spi_device * spi, const void * txbuf, unsigned n_tx, void * rxbuf, unsigned n_rx)

where,

spi – device with which data will be exchanged.

txbuf – data to be written.

n_tx – size of txbuf (in bytes).

rxbuf – buffer into which data will be read.

n_rx – size of rxbuf (in bytes).

SPI Driver – Implementation

Remove the device

Using the below API, you can unregister the slave device.

void spi_unregister_device(struct spi_device *spi)

where,

spi – a device that needs to be removed.

SPI Driver – Implementation

```
static void __exit test_spi_exit(void)
{
  if( test_spi_device )
  {
    spi_unregister_device( test_spi_device );    // Unregister the SPI slave
    pr_info("SPI driver Unregistered\n");
  }
}
```

SPI Driver – Handson

Please refer to the below connection diagram.

D1 (MOSI) – GPIO 20 (SP1_MOSI)

D0 (CLK) – GPIO 21 (SP1_SCK)

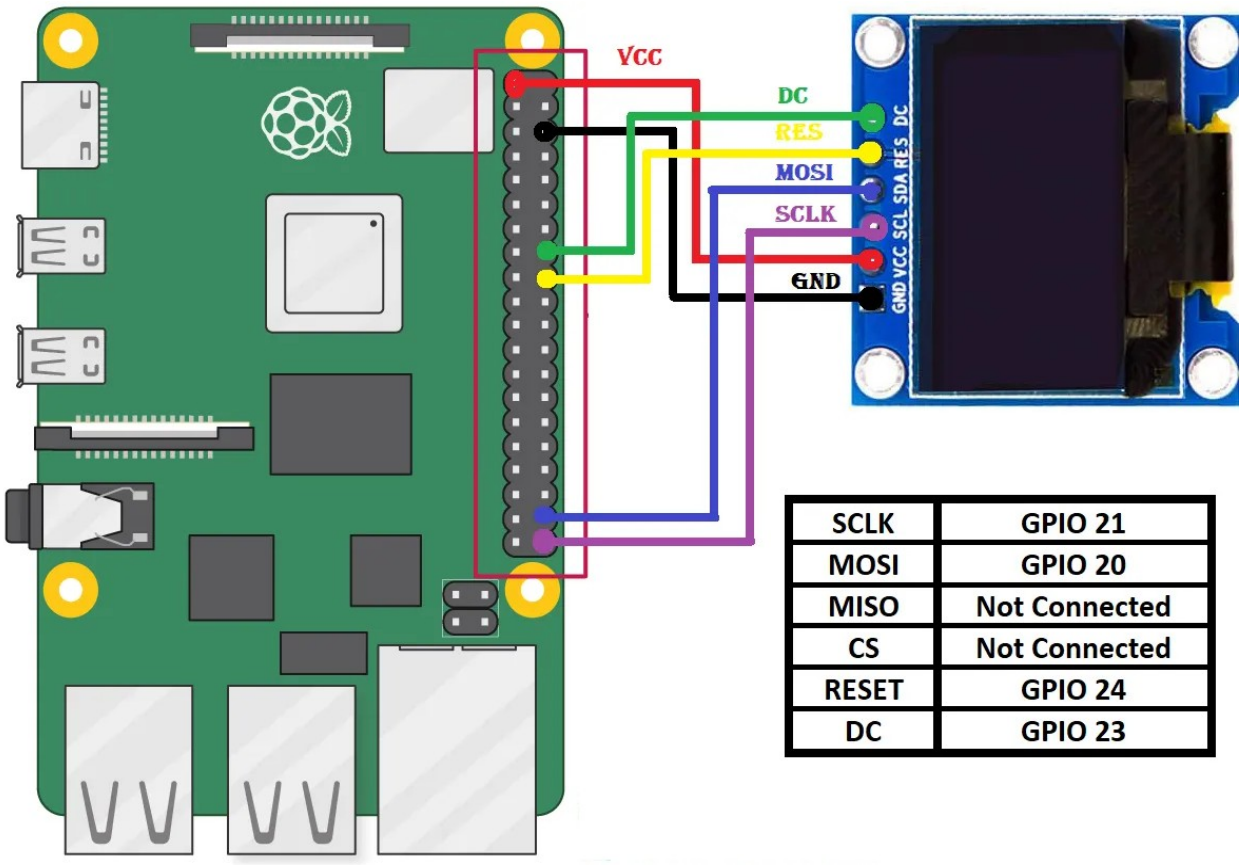
RES – GPIO 24

DC – GPIO 23

Vcc – 3.3V

GND – Ground

CS- GPIO16 (SPI1_CS0)



SCLK	GPIO 21
MOSI	GPIO 20
MISO	Not Connected
CS	Not Connected
RESET	GPIO 24
DC	GPIO 23

Enable the SPI using dtboverlay cmd:

sudo dtoverlay spi1-1cs cs0_spidev=disabled

User space tools

spidev

What does spidev do?

- Passes data between user space and SPI controller
- Collects buffers for TX/RX from user space application
- Hands off buffers to SPI controller driver
- Returns to user space when transfer is complete

User space tools

spidev

The sysfs node for the SPI device will include a child device node with a “dev” attribute that will be understood by udev or mdev.

For a SPI device with chip select C on bus B, you should see:

- **/dev/spidevB.C** - character special device, major number 153 with a dynamically chosen minor device number.
- **/sys/devices/…/spiB.C** - SPI device node will be a child of its SPI master controller.
- **/sys/class/spidev/spidevB.C** - created when the “spidev” driver binds to that device.

SPI Driver – Handson

There is a minimal core of SPI programming interfaces, focussing on using the driver model to connect controller and protocol drivers using device tables provided by board specific initialization code. SPI shows up in sysfs in several locations:

```
/sys/devices/.../CTLR ... physical node for a given SPI controller

/sys/devices/.../CTLR/spiB.C ... spi_device on bus "B",
     chipselect C, accessed through CTLR.

/sys/bus/spi/devices/spiB.C ... symlink to that physical
     .../CTLR/spiB.C device

/sys/devices/.../CTLR/spiB.C/modalias ... identifies the driver
     that should be used with this device (for hotplug/coldplug)

/sys/bus/spi/drivers/D ... driver for one or more spi*.* devices

/sys/class/spi_master/spiB ... symlink to a logical node which could hold
     class related state for the SPI master controller managing bus "B".
     All spiB.* devices share one physical SPI bus segment, with SCLK,
     MOSI, and MISO.

/sys/devices/.../CTLR/slave ... virtual file for (un)registering the
     slave device for an SPI slave controller.
     Writing the driver name of an SPI slave handler to this file
     registers the slave device; writing "(null)" unregisters the slave
     device.
     Reading from this file shows the name of the slave device ("(null)"
     if not registered).

/sys/class/spi_slave/spiB ... symlink to a logical node which could hold
     class related state for the SPI slave controller on bus "B".  When
     registered, a single spiB.* device is present here, possible sharing
     the physical SPI bus segment with other SPI slave devices.
```

At this time, the only class-specific state is the bus number ("B" in "spiB"), so those /sys/class entries are only useful to quickly identify busses.

PCI Driver - History

- ISA: Industry Standard Architecture (1981)
- PCI: Peripheral Component Interconnect
- An Intel-backed industry initiative (1992-9)
- Main goals:
  - Improve data-xfers to/from peripheral devices
  - Eliminate (or reduce) platform dependencies
  - Simplify adding/removing peripheral devices
  - Lower total consumption of electrical power

PCI Driver - History

- The PCI architecture was designed as a replacement for the ISA standard, with three main goals:
  - to get better performance when transferring data between the computer and its peripherals
  - to be as platform independent as possible
  - and to simplify adding and removing peripherals to the system.

- The PCI bus achieves better performance by using a higher clock rate than ISA; its clock runs at 25 or 33 MHz (its actual rate being a factor of the system clock), and 66-MHz and even 133-MHz implementations have recently been deployed as well.

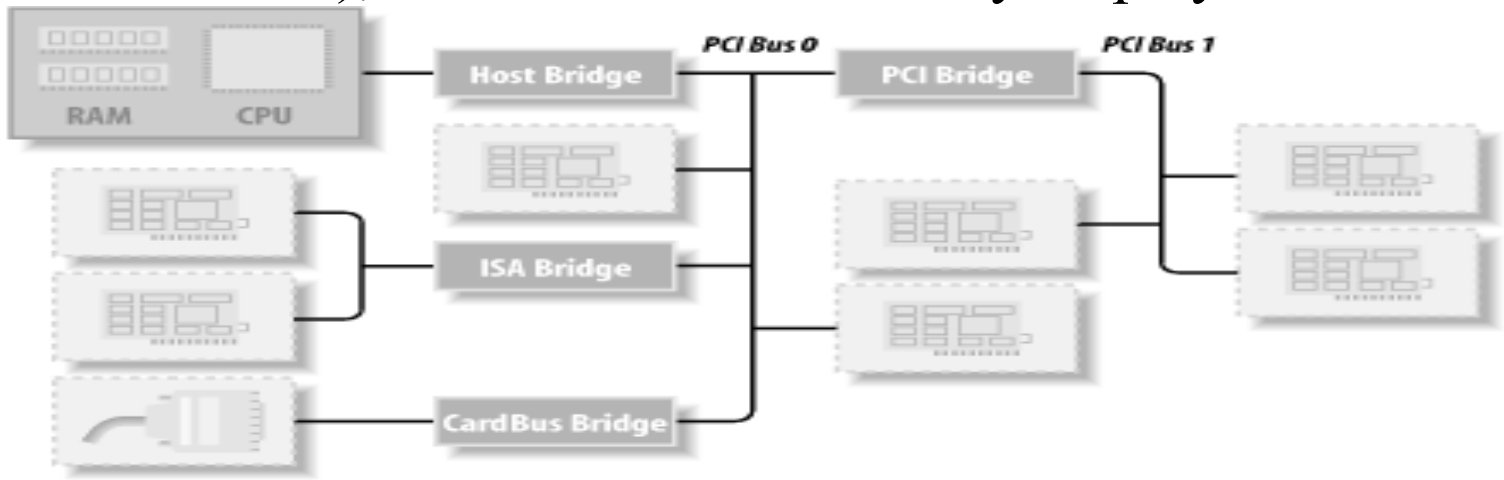
- It is equipped with a 32-bit data bus, and a 64-bit extension has been included in the specification.

- The device driver must be able to access configuration information in the device in order to complete initialization. This happens without the need to perform any probing.

PCI Driver - Addressing

- Each PCI peripheral is identified by a bus number, a device number, and a function number. The PCI specification permits a single system to host up to 256 buses, but because 256 buses are not sufficient for many large systems, Linux now supports PCI domains. Each PCI domain can host up to 256 buses. Each bus hosts up to 32 devices, and each device can be a multifunction board (such as an audio device with an accompanying CD-ROM drive) with a maximum of eight functions.

- When the hardware address is displayed, it can be shown as two values (an 8-bit bus number and an 8-bit device and function number), as three values (bus, device, and function), or as four values (domain, bus, device, and function); all the values are usually displayed in hexadecimal.



PCI Driver – lspci Command

- Linux scans PCI Configuration Space
- It builds a list of ‘pci_dev_struct’ objects
- It exports partial info using a ‘/proc’ file
- You can view this info using a command:

  $ /sbin/lspci

- Or you can directly view the /proc/bus/pci file:

  $ cat /proc/bus/pci

PCI Driver – lspci Command

1. Default Usage

By default it will display all the device information as shown below. The first field is the slot information in this format: [domain:]bus:device.function

In this example, since all the domain are 0, lspci will not display the domain.

```
# lspci
00:00.0 Host bridge: Intel Corporation 5500 I/O Hub to ESI Port (rev 13)
00:01.0 PCI bridge: Intel Corporation 5520/5500/X58 I/O Hub PCI Express Root Port 1 (rev 13)
00:09.0 PCI bridge: Intel Corporation 7500/5520/5500/X58 I/O Hub PCI Express Root Port 9 (rev 13)
00:14.0 PIC: Intel Corporation 7500/5520/5500/X58 I/O Hub System Management Registers (rev 13)
00:14.1 PIC: Intel Corporation 7500/5520/5500/X58 I/O Hub GPIO and Scratch Pad Registers (rev 13)
00:14.2 PIC: Intel Corporation 7500/5520/5500/X58 I/O Hub Control Status and RAS Registers (rev 13)
00:1a.0 USB controller: Intel Corporation 82801I (ICH9 Family) USB UHCI Controller #4 (rev 02)
00:1c.0 PCI bridge: Intel Corporation 82801I (ICH9 Family) PCI Express Port 1 (rev 02)
00:1d.0 USB controller: Intel Corporation 82801I (ICH9 Family) USB UHCI Controller #1 (rev 02)
00:1e.0 PCI bridge: Intel Corporation 82801 PCI Bridge (rev 92)
00:1f.0 ISA bridge: Intel Corporation 82801IB (ICH9) LPC Interface Controller (rev 02)
00:1f.2 IDE interface: Intel Corporation 82801IB (ICH9) 2 port SATA Controller [IDE mode] (rev 02)
01:00.0 Ethernet controller: Broadcom Corporation NetXtreme II BCM5709 Gigabit Ethernet (rev 20)
01:00.1 Ethernet controller: Broadcom Corporation NetXtreme II BCM5709 Gigabit Ethernet (rev 20)
03:00.0 RAID bus controller: LSI Logic / Symbios Logic MegaRAID SAS 2108 [Liberator] (rev 05)
06:03.0 VGA compatible controller: Matrox Electronics Systems Ltd. MGA G200eW WPCM450 (rev 0a)
```

PCI Driver – lspci Command

Dump PCI Info in Different Format

If you want to pass the output of the lspci command to a shell script, you may want to use -m option (or -mm option) as shown below.

This option is also helpful when you want to view the subsystem information. For example, for the RAID controller, the default output just says that is is using LSI Logic RAID controller. But, the following output displays the subsystem, which is DELL PERC H700 Integrated RAID controller system.

```
# lspci -m

00:00.0 "Host bridge" "Intel Corporation" "5500 I/O Hub to ESI Port" -r13 "Dell"
"PowerEdge R610 I/O Hub to ESI Port"

00:09.0 "PCI bridge" "Intel Corporation" "7500/5520/5500/X58 I/O Hub PCI Express Root
Port 9" -r13 "" ""

00:14.0 "PIC" "Intel Corporation" "7500/5520/5500/X58 I/O Hub System Management
Registers" -r13 "" ""

00:1a.0 "USB controller" "Intel Corporation" "82801I (ICH9 Family) USB UHCI
Controller #4" -r02 "Dell" "PowerEdge R610 USB UHCI Controller"

00:1f.0 "ISA bridge" "Intel Corporation" "82801IB (ICH9) LPC Interface Controller" -
r02 "Dell" "PowerEdge R610 82801IB (ICH9) LPC Interface Controller"

00:1f.2 "IDE interface" "Intel Corporation" "82801IB (ICH9) 2 port SATA Controller
[IDE mode]" -r02 -p8f "Dell" "PowerEdge R610 SATA IDE Controller"

01:00.0 "Ethernet controller" "Broadcom Corporation" "NetXtreme II BCM5709 Gigabit
Ethernet" -r20 "Dell" "PowerEdge R610 BCM5709 Gigabit Ethernet"

03:00.0 "RAID bus controller" "LSI Logic / Symbios Logic" "MegaRAID SAS 2108
[Liberator]" -r05 "Dell" "PERC H700 Integrated"

06:03.0 "VGA compatible controller" "Matrox Electronics Systems Ltd." "MGA G200eW
WPCM450" -r0a "Dell" "PowerEdge R610 MGA G200eW WPCM450"
```

PCI Driver – lspci Command

Output in Tree Format

The -t option will display the output in tree format with information about bus, and how devices are connected to those buses as shown below. The output will be only using the numerical ids.

```
# lspci -t

-[0000:00]-+-00.0

            +-01.0-[01]--+-00.0

            |            \-00.1

            +-03.0-[02]--+-00.0

            |            \-00.1

            +-07.0-[04]--

            +-09.0-[05]--

            +-14.0

            +-14.1

            +-1c.0-[03]----00.0

            +-1d.0

            +-1e.0-[06]----03.0

            +-1f.0
```

PCI Driver – lspci Command

Detailed Device Information

If you want to look into details of a particular device, use -v to get more information. This will display information about all the devices. The output of this command will be very long, and you need to scroll down and view the appropriate section.

For additional level for verbosity, you can use -vv or -vvv.

In the following example, I've given output of only the RAID controller device.

```
# lspci -v

 03:00.0 RAID bus controller: LSI Logic / Symbios Logic MegaRAID SAS 2108 [Liberator]
(rev 05)

        Subsystem: Dell PERC H700 Integrated

        Flags: bus master, fast devsel, latency 0, IRQ 16

        I/O ports at fc00 [size=256]

        Memory at df1bc000 (64-bit, non-prefetchable) [size=16K]

        Memory at df1c0000 (64-bit, non-prefetchable) [size=256K]

        Expansion ROM at df100000 [disabled] [size=256K]

        Capabilities: [50] Power Management version 3
```

PCI Driver – lspci Command

Display Device Codes in the Output

If you want to display the PCI vendor code, and the device code only as the numbers, use -n option. This will not lookup the PCI file to get the corresponding values for the numbers.

```
# lspci -n

01:00.1 0200: 14e4:1639 (rev 20)

02:00.0 0200: 14e4:1639 (rev 20)

02:00.1 0200: 14e4:1639 (rev 20)

03:00.0 0104: 1000:0079 (rev 05)

06:03.0 0300: 102b:0532 (rev 0a)
```

If you want to display both the description and the number, use the option -nn as shown below.

```
# lspci -nn
```

PCI Driver – lspci Command

Lookup a Specific Device

When you know the slot number in the domain:bus:slot.func format, you can query for a particular device as shown below. In the following example, we didn't specify the domain number, as it is 0, which can be left out.

```
# lspci -s 03:00.0

03:00.0 RAID bus controller: LSI Logic / Symbios Logic MegaRAID SAS 2108 [Liberator]
(rev 05)
```

When you know the device number in the vendor:device format, you can query for a particular device as shown below.

```
# lspci -d 1000:0079

03:00.0 RAID bus controller: LSI Logic / Symbios Logic MegaRAID SAS 2108 [Liberator]
(rev 05)
```

If you know only either the vendor id, or the device id, you can omit the other id. For example, both the following command will return the same output as the above.

PCI Driver – lspci Command

Display Kernel Drivers

This is very helpful when you like to know the name of the kernel module that will be handling the operations of a particular device. Please note that this option will work only on Kernel 2.6 version and above.

```
# lspci -k

00:1f.2 IDE interface: Intel Corporation 82801IB (ICH9) 2 port SATA Controller [IDE
mode] (rev 02)

        Subsystem: Dell PowerEdge R610 SATA IDE Controller

        Kernel driver in use: ata_piix

        Kernel modules: ata_generic, pata_acpi, ata_piix

02:00.0 Ethernet controller: Broadcom Corporation NetXtreme II BCM5709 Gigabit
Ethernet (rev 20)

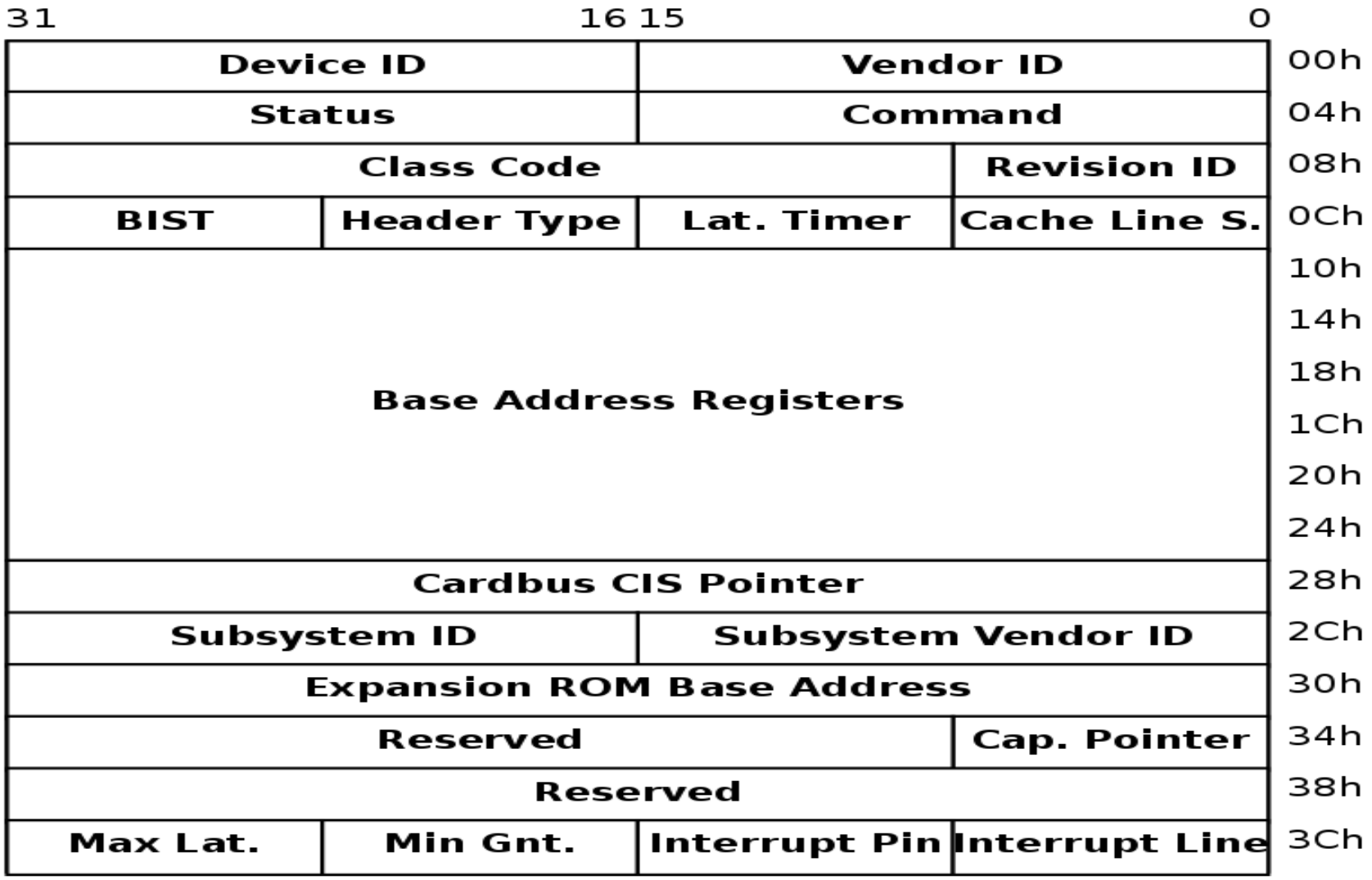
        Subsystem: Dell PowerEdge R610 BCM5709 Gigabit Ethernet

        Kernel driver in use: bnx2
```

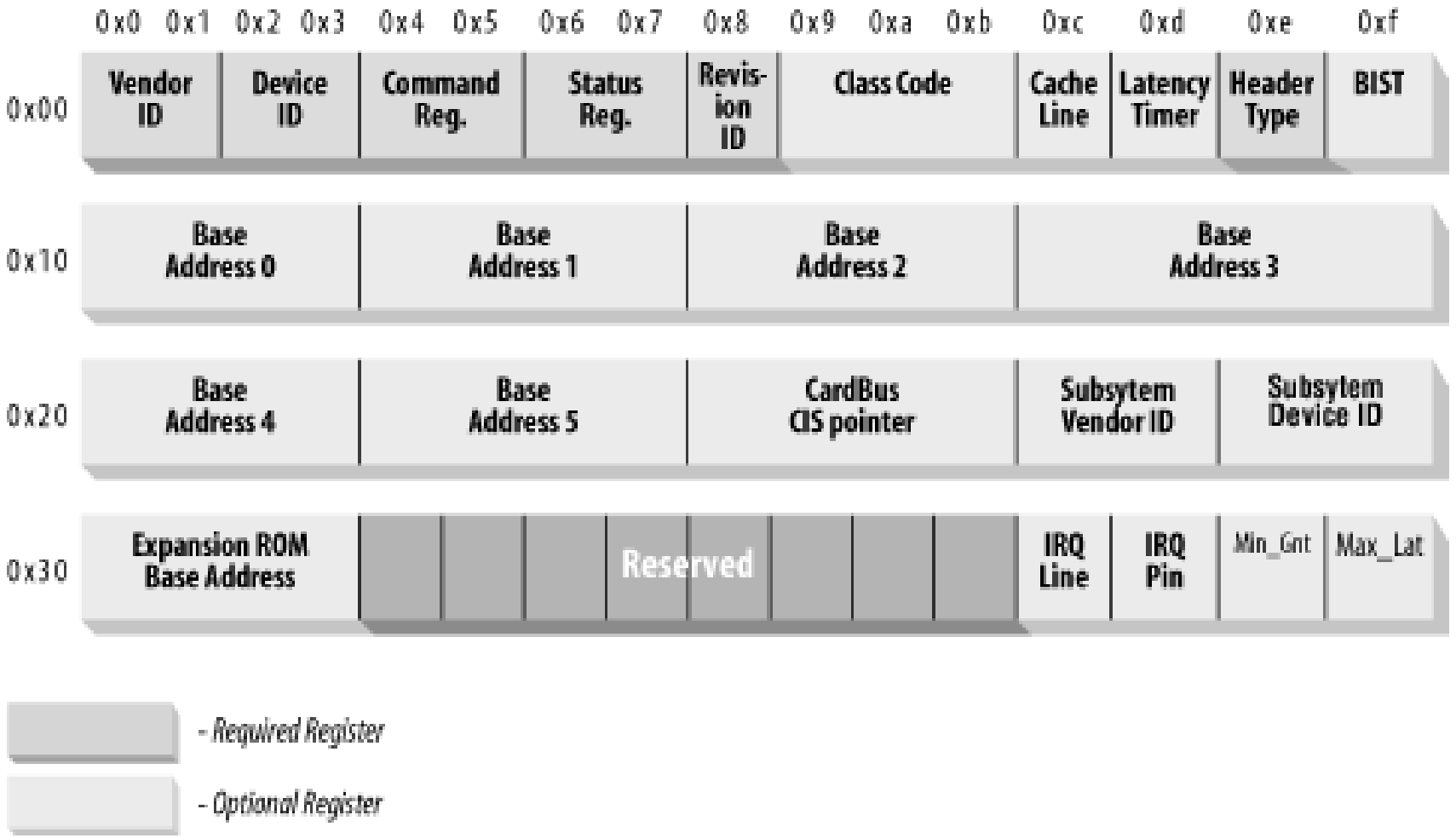
PCI Driver – Configuration Space

- PCI devices have a set of registers referred to as configuration space and PCI Express introduces extended configuration space for devices. Configuration space registers are mapped to memory locations. Device drivers and diagnostic software must have access to the configuration space, and operating systems typically use APIs to allow access to device configuration space. When the operating system does not have access methods defined or APIs for memory mapped configuration space requests, the driver or diagnostic software has the burden to access the configuration space in a manner that is compatible with the operating system's underlying access rules. In all systems, device drivers are encouraged to use APIs provided by the operating system to access the configuration space of the device.

PCI Driver – Configuration Space



PCI Driver – Configuration Space



Thank you